

7-dars: Feature Analysis in SML

Case:

- 50+ featurelar bor
- Lekin xamma featurelar xam foydali bo'lmaydi.
- Foydasiz features:

↳ Irrelevant → add noise, reduce accuracy

• (Gol → Ovgat)

↳ Highly correlated → redundant information

• (jami gol va xar bir oyindagi gol)

↳ Nonlinear or complex → not captured by simple models

• (Gol → futbolchi larni charchashi)

- FA dagi asosiy maqsad nima bo'lishi kerak?

↳ Faqatgina "target" qiymatga to'g'ri ta'sir qiladigan featurelarni aniqlash.

↳ Irrelevant featurelarni aniqlash va tashlab yuborish.

↳ Improve model performance, speed, and interpretability.

- Interpretability (shaffor: Trust & Transparency)
- Medical model: shuncha eng muhimi, featurelarga tushuntirish.

◦ Black - Box (tushunarsiz)

- FA input qiymatlarni predictive powerini aniqlash va ularni "target" qiymatiga ta'sirini tushiradigan ML jarayoni (example: Feature engineering)

◦ FA SML da

- Which features most affect the "target"?
- Are some features redundant or correlated with others?
- Should we transform any features?

◦ Feature muhimligini aniqlash:

- Correlation with target

↳ Numeric: Pearson & Spearman

↳ Categorical: ANOVA, chi-square

- Correlation with other features

↳ Remove highly correlated (multicollinearity)

-- model ikki lani' b qoladi qaysi biri target qiymatga ta'sir qiladi?

↳ Avoid redundant info --- yaxshi generalization qila olmaydi.

- Feature importance from models

↳ Linear models: Coefficients

↳ Trees: Gini importance, SHAP

Input vs Output

Input vs Input

➔ Bunda "features" ni algorithm/ ergodi ham aniqlash mumkinligi keliib chiqadi.

o Performance impact

↳ Train model with/without feature →
"check test metrics" → (R^2 , RMSE, accuracy)

CODING PART

I. # Correlation with target : Numeric features
→ Pearson & Spearman.

```
import pandas as pd
```

```
numeric_features = df.select_dtypes(include=["int64",  
: target = df["x6"] "float64"]).drop(columns=["x6"])
```

```
target = df["x6"]
```

```
# Pearson correlation
```

```
pearson_corr = df[numeric_features.columns].  
corrwith(target)
```

```
print("Pearson Correlation with x6:\n",  
      pearson_corr)
```


No. • CorC) Bln fargi => Bu barcha nuqsona
feature larni bir-biri bln solishtirish.
radi.

corr + with = "..."
Correlation hisoblagan
(target)

Spearman correlation

spearman_corr = df[numerical_features.columns]

corrwith(target, method="spearman")

print("\n Spearman Correlation with x6: \n",
spearman_corr)

II. Correlation with target: Categorical features
→ ANOVA, chi-square

import pandas as pd
from scipy.stats import f_oneway

--- Select categorical features ---

categorical_features = df.select_dtypes(include=[
"object"]).columns

--- Store ANOVA results ---

anova_results = {}

for col in categorical_features:

groups = [df[df[col] == val]['x6']
for val in df[col].unique()]

```

if len(groups) > 1:
    f_stat, p_val = f_oneway(*groups)
    anova_results[60] = {"F-statistic":
        f_stat, 'p-value': p_val}

```

```

anova_df = pd.DataFrame(anova_results).T
sort_values('p-value')

```

```

print("ANOVA Results for Categorical Features vs
      x6:\n")

```

```

print(anova_df)

```

III. Correlation with Others

```

import seaborn as sns

```

```

import matplotlib.pyplot as plt

```

```

plt.figure(figsize=(10,8))

```

```

sns.heatmap(df[numeric_features.columns].corr(),
            annot=True, cmap="coolwarm")

```

```

plt.title("Feature-to-Feature Correlation")

```

```

plt.show()

```

Example: drop features with correlation > 0.9


```

corr_matrix = df[numeric_features.
columns].corr().abs()
upper = corr_matrix.where(np.triu(np.ones(
(corr_matrix.shape), k=1).astype(bool))
to_drop = [column for column in upper.columns
if any(upper[column] > 0.9)]
print("Highly correlated features to drop:",
to_drop)

```

plotly Express

```

import plotly.express as px
import numpy as np
import pandas as pd

```

--- Compute correlation matrix ---

```

corr_matrix = df[numeric_features.columns].
corr()

```

--- Interactive Heatmap with Plotly ---

```

fig = px.imshow(
    corr_matrix,
    text_auto=True, # Show correlation numbers
    color_continuous_scale="RdBu_r",
    origin="upper",

```

```

labels = dict(x = "Features", y = "Features",
              color = "Correlation")
title = "Feature - to - Feature Correlation (
        Numeric Features)"
)

```

```
fig. update_axes (side = "top")
```

```
fig. show()
```

--- Identify highly correlated features ($r > 0.9$) to consider dropping ---

```
upper = corr_matrix.abc().where(np.triu(
    np.ones(corr_matrix.shape), k=1).
    astype(bool))
```

```
to_drop = [col for col in upper.columns
            if any(upper[col] > 0.9)]
```

```
print("Highly correlated features to drop:",
      to_drop)
```

(O'xkita correlated bo'lganidan qaysi birini
olib qolib qaysi birini tashlab qabozishi
uncheon "Domain knowledge"dan foydalanamiz.

2. Holatda: U featureni boshqa featurelar
b/y "correlationini" ko'rish k-k. Ya'ni
qolgan/ b/y qanday o'logani bor ya'ni
analiz qilinadi.

Bekorga bu bo'lim = umumiy analiz tuzasda)

Bu analiz "Feature Engineering" dan keyin bo'lish kerak.

Summary from 1 and 2 qanday esa

- Agar biron feature predictive bo'lsa lekin "highly correlated" bo'lsa tashlab yubormaslik kerak.
- Uning o'rniga ratio or feature engineering qilish kerak.
- Tashlab yuborishda ham correlation a and b bo'lsa qaysi biri target qiymatga predictive ekanligidan kelib chiqib tashlab yuboriladi.

FEATURE IMPORTANCE FROM MODELS

#linear

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

X = numeric_features

y = target

scaler = StandardScaler()


```
X_scaled = scaler.fit_transform(X)
```

```
lr = LinearRegression()
```

```
lr.fit(X_scaled, y)
```

```
coefficients = pd.Series(lr.coef_, index = X.columns)
```

```
coefficients.sort_values(key = abs, ascending = False)
```

```
print("Linear Regression Feature Coefficients:\n",  
      coefficients)
```

```
# Tree
```

```
from sklearn.ensemble import RandomForestRegressor  
import pandas as pd
```

```
rf = RandomForestRegressor(random_state = 42)
```

```
rf.fit(X, y)
```

```
# Feature Importance
```

```
feat_importance = pd.Series(rf.feature_  
                             importances_, index = X.columns).sort_  
                             values(ascending = False)
```

```
print("Random Forest Feature Importance:\n",  
      feat_importance)
```

Buni 2ta so'latda qo'llasak bo'ladi. 1- FA qilib filtr qilmasdan oldin bir ko'rib olgani. 2- FE qilib keyin qo'llasak ham bo'ladi. FE qilgandan keyingi featurelar qanday o'rinda turi'ldi o'sha orqali tekshirish bo'ladi.

PERFORMANCE IMPACT

lagad : Featurelarni olib qolish / tashlab
yuborishni ta'sirini ko'rish.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error
```

Train / test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.3, random_state = 42)
```

Train full model

```
rf.fit(X_train, y_train)
y_pred_full = rf.predict(X_test)
full_rmse = mean_squared_error(y_test, y_pred_full) ** 0.5
```

```
full_r2 = r2_score(y_test, y_pred_full)
```

Drop one feature

```
X_train_reduced = X_train.drop(columns = ['shots'])
X_test_reduced = X_test.drop(columns = ['shots'])
rf.fit(X_train_reduced, y_train)
y_pred_reduced = rf.predict(X_test_reduced)
reduced_rmse = mean_squared_error(y_test, y_pred_reduced) ** 0.5
```


$\text{reduced_r2} = \text{r2_score}(y_test, y_pred_reduced)$

```
print(f"Full model  $R^2$ : {full_r2:.3f}, RMSE:  
      {full_rmse:.3f}")
```

```
print(f"Reduced model  $R^2$ : {reduced_r2:.3f},  
      RMSE: {reduced_rmse:.3f}")
```

plotly studio (MacOS) versiyani yuklab
ishlatish, o'rganish va projectga tatbiq
qilish kerak!